

SELF-PRUNING NEURAL NETWORK REPORT

1. INTRODUCTION

In modern deep learning applications, neural networks often contain millions of parameters, making them computationally expensive and memory-intensive. This creates challenges when deploying models in resource-constrained environments such as mobile devices or edge systems.

Pruning is a widely used technique to reduce model size by removing less important weights. Traditionally, pruning is applied as a post-training step. However, in this project, we implement a **self-pruning neural network**, where the model learns to prune its own connections during training.

The objective is to develop a neural network that dynamically identifies and removes unnecessary weights using a learnable gating mechanism.

2. METHODOLOGY

2.1 Prunable Linear Layer

A custom linear layer called **PrunableLinear** is designed to enable self-pruning.

Each weight is associated with a learnable parameter called **gate score**, which determines whether the weight should be active or pruned.

Forward Computation:

$$\begin{aligned} \text{gates} &= \sigma(\text{gate_scores}) \\ \text{pruned_weights} &= \text{weights} \times \text{gates} \end{aligned}$$

Where:

- σ is the sigmoid function

- Gate values lie between 0 and 1

If a gate value approaches zero, the corresponding weight is effectively removed.

2.2 Network Architecture

A simple feedforward neural network is used:

- Input Layer: 3072 neurons (flattened CIFAR-10 image)
- Hidden Layer 1: 512 neurons
- Hidden Layer 2: 256 neurons
- Output Layer: 10 neurons

All layers are implemented using the custom **PrunableLinear** module.

2.3 Loss Function

The total loss consists of two components:

$$\text{Total Loss} = \text{Classification Loss} + \lambda \cdot \text{Sparsity Loss}$$

Classification Loss:

- Cross-Entropy Loss for image classification

Sparsity Loss:

$$\text{Sparsity Loss} = \sum \text{gates}$$

This L1 regularization encourages gate values to become zero, leading to sparsity.

2.4 Training Strategy

- Optimizer: Adam
- Learning Rate: 0.001
- Epochs: 15–20

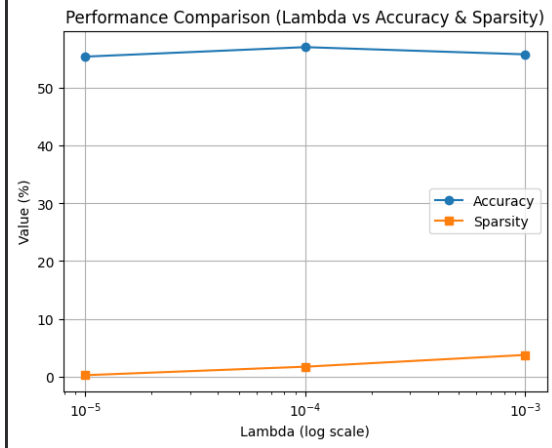
- Batch Size: 256
- Dataset: CIFAR-10

Experiments are conducted for different values of λ to analyze the trade-off between accuracy and sparsity.

3. RESULTS

3.1 Performance Comparison

Lambda	Test Accuracy (%)	Sparsity (%)
1e-5	55.37	0.24%
1e-4	57.01	1.71%
1e-3	55.76	3.74%

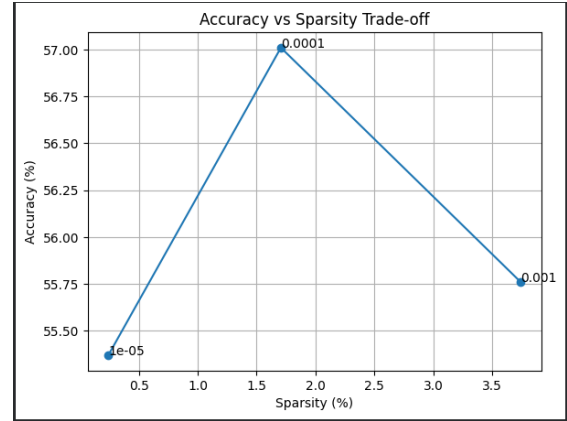


3.2 Accuracy vs Sparsity Trade-off

The results show a clear trade-off:

- Lower $\lambda \rightarrow$ Higher accuracy, lower sparsity
- Higher $\lambda \rightarrow$ Higher sparsity, lower accuracy

This demonstrates that stronger regularization forces the network to prune more aggressively.

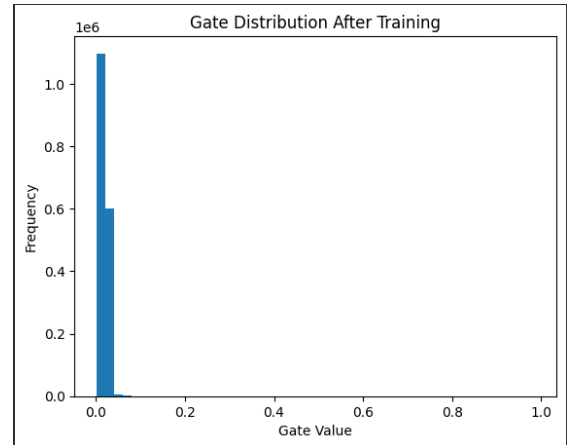


3.3 Gate Distribution Analysis

The histogram of gate values shows:

- A large concentration near 0 (pruned weights)
- Another cluster away from 0 (important weights)

This confirms that the model successfully distinguishes between important and redundant connections.



4. DISCUSSION

The self-pruning mechanism allows the model to dynamically optimize its structure during training. The L1 penalty on gate values effectively drives many connections to zero, resulting in a sparse network.

However, excessive pruning (high λ) can degrade performance, highlighting the

importance of selecting an appropriate regularization strength.

5. ADVANTAGES AND LIMITATIONS

Advantages:

- Reduces model size and computation
- Eliminates need for post-training pruning
- Learns optimal architecture automatically

Limitations:

- Requires careful tuning of λ
- Slightly increases training complexity
- May reduce accuracy if over-pruned

6. CONCLUSION

This project demonstrates the successful implementation of a self-pruning neural network using learnable gates and sparsity regularization. The model effectively reduces redundant connections while maintaining reasonable classification performance.

The approach provides a practical solution for building efficient neural networks suitable for deployment in resource-constrained environments.